

Algorithms for Data Science

Exercise 5

(with Solutions)

See the cheatsheet for valuable remarks!

For simplicity, in some of the tasks we consider only the keys of the key-value pairs.

Task 5.1: *Simple Tasks: Hash and BST*

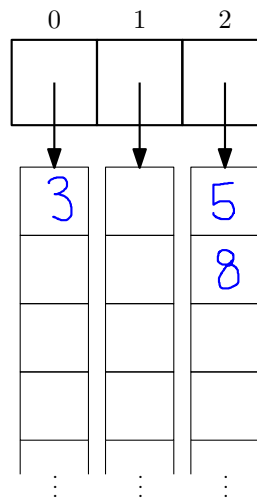
- (a) Add the following keys in the given order into a hash table of size 3 using the hash function

$$h(k) = k \mod 3$$

keys: 3, 5, 8

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.

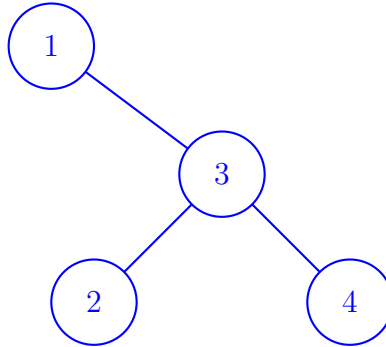
Solution:



- (b) Add the following keys in the given order to an initially empty binary search tree:

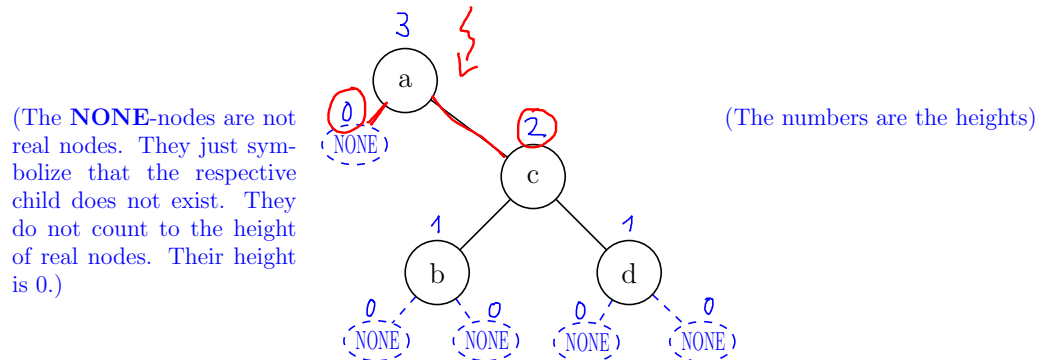
1, 3, 4, 2

Solution:



- (c) Compute the height for each node and check whether the AVL-invariant is satisfied. *Recall that the height of a node v is the number of nodes on a longest path starting at v within the subtree of v .*

Solution:



- (d) Design an algorithm **FindMin**(*root*) in pseudocode that returns the minimum key of the binary search tree rooted at *root* (whose root node is *root*). You can assume that the tree contains at least one element.

Solution:

```

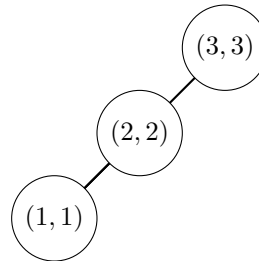
findMinKey(root)
  node = root
  while node.left ≠ NONE do
    node = node.left
  return node.key
  
```

- (e) Draw a binary search tree that additionally satisfies the heap property with respect to the priorities containing the following key-priority pairs:

(1, 1), (2, 2), (3, 3)

The tree does not need to be a complete binary tree as required by our definition of a heap. It just have to be a binary tree that satisfies the BST invariant and heap invariant at the same time.

Solution:



Task 5.2: Hash Tables

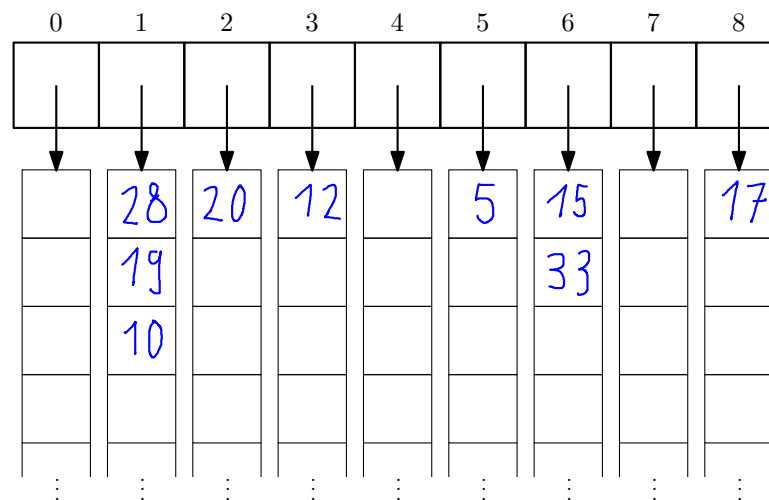
Add the following keys in the given order into a hash table of size 9 using the hash function

$$h(k) = k \bmod 9.$$

keys: 5, 28, 19, 15, 20, 33, 12, 17, 10

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.

Solution:

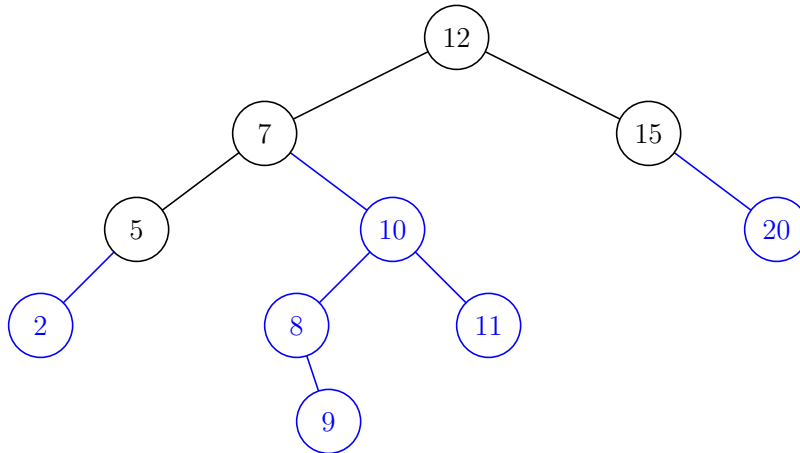


Task 5.3: Binary Search Trees

- (a) Given the binary search tree below, add the following keys in the given order:

10, 20, 8, 5, 2, 9, 11

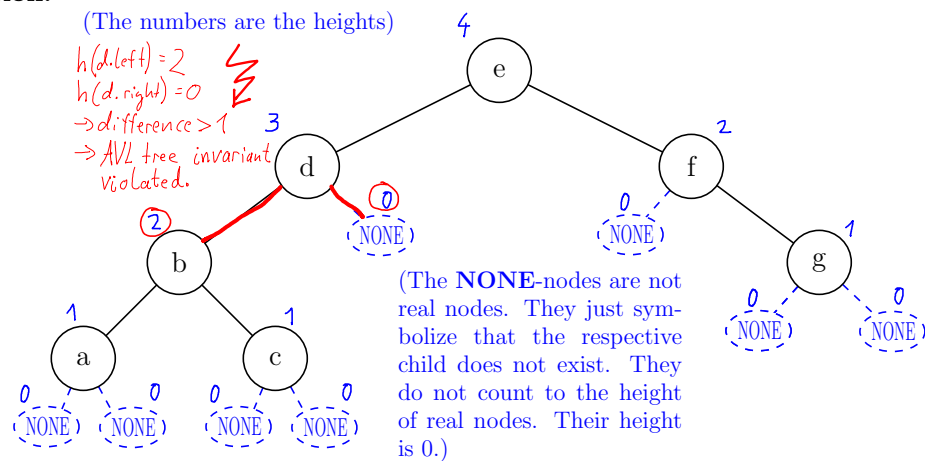
Solution:



Note that the key 5 is already in the tree, so the tree does not change (and only the associated value would be updated).

- (b) Compute the height for each node and check whether the AVL-invariant is satisfied.

Solution:



- (c) Design an algorithm $\text{Add}(\text{root}, k, v)$ in pseudocode that adds the key-value pair (k, v) to the binary search tree rooted at root . (If the tree is empty, $\text{root} = \text{NONE}$.) The added node should be returned.

Solution: We will use the following auxiliary function:

```
CreateNewNode( $k, v$ )  
┌  $w = \text{new node}$   
┌  $w.\text{key} = k$   
┌  $w.\text{value} = v$   
┌  $w.\text{left} = \text{NONE}$   
┌  $w.\text{right} = \text{NONE}$   
└ return  $w$ 
```

One possible solution using recursion:

```
Add( $\text{root}, k, v$ )  
┌ return AddRec( $\text{root}, \text{false}, \text{NONE}, k, v$ )  
AddRec( $\text{node}, \text{isLeftChild}, \text{parent}, k, v$ )  
┌ if  $\text{node} == \text{NONE}$  then  
┌  $\text{node} = \text{CreateNewNode}(k, v)$   
┌ if  $\text{parent} \neq \text{NONE}$  then  
┌ if  $\text{isLeftChild}$  then  
┌  $\text{parent.left} = \text{node}$   
┌ else  
┌  $\text{parent.right} = \text{node}$   
└ return  $\text{node}$   
else  
┌ if  $k < \text{node.key}$  then  
┌ return AddRec( $\text{node.left}, \text{true}, \text{node}, k, v$ )  
┌ else if  $\text{node.key} > k$  then  
┌ return AddRec( $\text{node.right}, \text{false}, \text{node}, k, v$ )  
┌ else  
┌  $\text{node.value} = v$   
└ return  $\text{node}$ 
```

And another recursive solution, without extra parameters:

```
Add(root,k,v)
| if root == NONE then
|   root = CreateNewNode(k,v)
|   return root
| else
|   return AddRec(root, k, v)
|
AddRec(root,k,v)
| if k < root.key then
|   if root.left == NONE then
|     w = CreateNewNode(k,v)
|     w.parent = root
|     root.left == w
|   else
|     w = AddRec(root.left, k, v)
|   return w
| else if root.key < k then
|   if root.right == NONE then
|     w = CreateNewNode(k,v)
|     w.parent = root
|     root.right == w
|   else
|     w = AddRec(root.right, k, v)
|   return w
| else
|   root.value = v
|   w = root
| return w
```

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 5.4: Which Data Structure?

For each of the following scenarios, propose the most appropriate data structure from our lecture (in terms of memory and time complexity). Explain your choice.

- (a) You want to keep track of all winning lottery numbers of a special lottery where the numbers have an unrealistic huge number of digits, say, several billions. You want to store each number together with its date, and you also want to check whether a given number has won in the past. The operations should be as fast as possible in practice and the memory usage not too large.

Solution: Correct answer: Hash table.

Explanation: The lottery numbers are random and therefore the average access time is constant with high probability; hence, very fast in practice. (Note that by randomly picking a hash

function from a universal hash family, we could keep the time constant—with high probability—also in the case of non-random lottery numbers.) Other data structures are worse. Arrays indexed by keys take too much memory and are therefore infeasible. Infeasible are also find-union, stack, queue, and heap as they do not provide operations to check if a specific number has been added to them. The remaining data structures covered in the lecture (AVL trees, linked lists, sorted/unsorted arrays) have much longer operation times in practice (not worst case) and are impractical also due to the fact that each comparison of the keys consists of comparing billions of digits. In a hash table, we have to compare the digits only when the hash functions are the same.

- (b) Upon registering on a social media platform, each user gets an automatically generated ID number. Your job is to maintain the data of all users that deleted their accounts. Since the data of deleted user accounts is (officially) kept only for legal reasons, the time to access and read such data does not matter. However, the insertion time is very important as the platform wants to minimize the total worst-case computational time (and thus the actual cost) spent on all the users that left.

Solution: Correct answer: Unsorted (dynamic) array (or queue, or stack, or an unsorted variant of the linked list where we always insert at the end).

Explanation: The best possible total time to add n users is $\Theta(n)$. This time is achieved with the unsorted array, queue, stack and linked list because they need only $O(1)$ (amortized) time for a single add operation. In other data structures (sorted array, heap, AVL tree, hash table), the time for a single add operation is at least $\Omega(\log n)$ in the worst (and amortized) case, making the total worst case time at least $\Theta(n \log n)$. Note that hash tables achieve $O(1)$ expected time per operation—which is sufficient in practice—but for a carefully crafted sequence of keys, a single operation can take $\Theta(n)$ in the worst case, resulting in a total worst-case time of $\Theta(n^2)$.

Task 5.5: Heap and BST

- (a) The heap introduced in the lecture is actually called a *max*-heap, as its name-giving operations `FindMax()` and `RemoveMax()` are with respect to the largest element value inside the heap.

Design a *min*-heap using a max-heap as a black box! That is, implement a data structure that has only the following operations and that internally uses a max-heap:

- `Init()`: Initializes an empty min-heap.
- `Add( $v$ )`: Adds the value v to the heap
- `FindMin()`: Returns the minimum value in the heap.
- `RemoveMin()`: Deletes an element with the minimum value in the heap and returns this value.

The running times should be asymptotically the same as the corresponding operations in the max-heap. You may assume that the elements are numbers and that the array underlying the max-heap is dynamic.

Solution: Idea: Just multiply incoming numbers by -1 and again by -1 before returning them.

```
minHeap.Init()
└─ maxHeap.Init()
```

```
minHeap.Add( $v$ )
└─ maxHeap.Add( $-v$ )
```

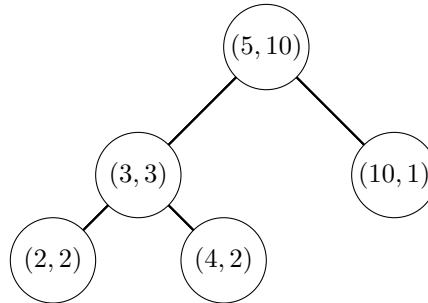
```
minHeap.findMin()
└─ return -maxHeap.findMax()
```

```
minHeap.removeMin()
└─ return -maxHeap.removeMax()
```

- (b) Draw a binary search tree that additionally satisfies the heap property with respect to the values containing the following key-value pairs:

$(3, 3), (2, 2), (4, 2), (10, 1), (5, 10)$

Solution: Start with the root of the tree which has to contain the pair with maximum value (heap property). By the binary search tree invariant, you know which nodes have to lie to the left and which ones to the right to the root (compare the keys). Recursively apply the two rules. The resulting (unique) tree:



- (c) Design an algorithm **Successor**(*root*, *k*) in pseudocode that returns the smallest key that is larger than *k* among the keys of the binary search tree rooted at *root*. You can assume that *k* and its successor exist in the tree.

Solution: The algorithm works in two steps. First we find a node (possibly the root) with the key *k*, or—if it does not exist—the empty child where an element with key *k* would be inserted. Secondly, we look for the successor of that node—see **SuccessorOfNode**(*node*). There are two cases. If the node has a right child, then we look for the smallest key in the subtree of its right child, hence, we call **findMin**..() for that subtree. If the node does not have a right child, then we look for the first grandparent with larger key; hence, we do a **findMin**..() in the tree above. We also consider the case that there is no key larger than *k*.

```

findMinKeyNode(node)
    while node.left ≠ NONE do
        | node = node.left
    return node

findMinKeyNodeAbove(node)
    while node.parent ≠ NONE and node.parent.left ≠ node do
        | node = node.parent
    return node.parent

SuccessorOfNode(node)
    if node.right == NONE then
        | return findMinKeyNodeAbove(node)
    return findMinKeyNode(node.right)
  
```

```

Successor(root, k)
    node = root
    par = NONE
    while node ≠ NONE and node.key ≠ k do
        par = node
        if k < node.key then
            | node = node.left
        else
            | node = node.right
    // The first if handles the (optional) case that the key k does not
    // exist in the tree.
    if node == NONE then
        if par == NONE then
            | return FAIL
        else if k < par.key then
            | return par.key
        | node = par
    node = SuccessorOfNode(node)
    if node == NONE then
        | return FAIL
    return node.key

```

- (d) Design an algorithm **SortedOutput**(*root*) in pseudocode that prints all the keys of the binary search tree rooted at *root* in sorted order. The running time should be linear in the number of elements. If you visit a node *node*, you can print its key by calling **print**(*node.key*).

Solution: A straightforward idea is to first find the node with the minimum key and print its key. Then, repeatedly, to look for the next successor using the auxiliary operations from the preceding task. The time of the solutions is linear in n as we walk along each edge at most twice: once down and once up.

```

SortedOutput(root)
    node = findMinKeyNode(root)
    while node ≠ NONE do
        | print(node.key)
        | node = SuccessorOfNode(node)

```

But there is a much simpler recursive solution:

```

SortedOutput(root)
    if root ≠ NONE then
        | SortedOutput(root.left)
        | print(root.key)
        | SortedOutput(root.right)

```

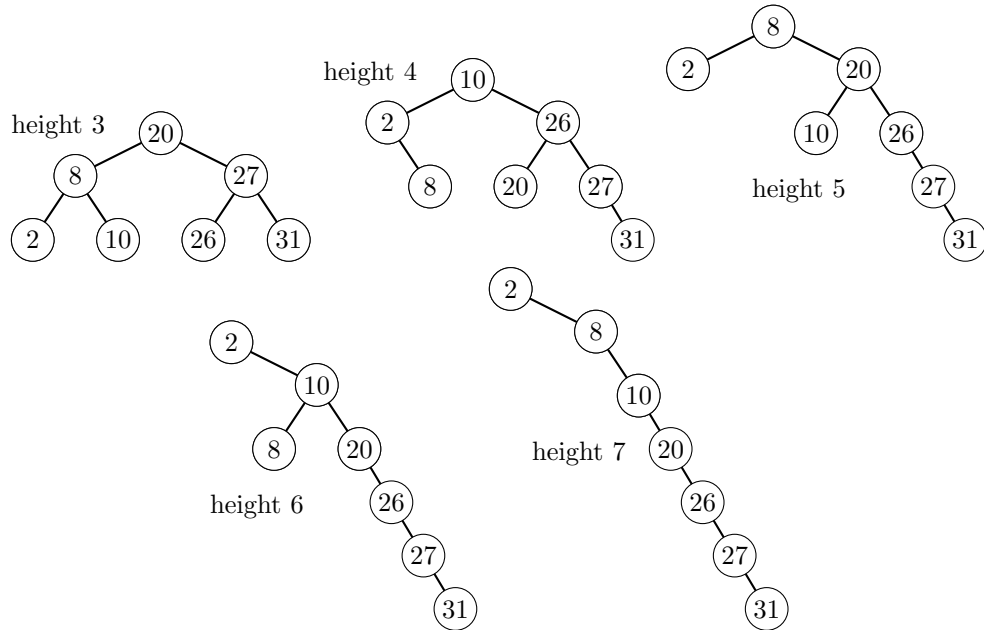
The time is linear in n , since we call the function at most once for every node and at most $2n$ for the value NONE (as every node has at most two NONE children).

(e) Draw binary search trees of heights 3, 4, 5, 6 and 7 each containing the following keys:

2, 8, 10, 20, 26, 27, 31 .

Recall that the height of a tree is the number of nodes on a longest path from the root.

Solution: For the heights 4 to 7 there is more than one solution.



Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 5.6: Interval Trees

Design a data structure that maintains a dynamic set of intervals and allows the following operations, each running in $O(\log n)$ time, where n is the current number of intervals stored in the data structure.

1. **Search** $([a, b])$: returns true if your data structure contains the interval $[a, b]$, otherwise returns false.
2. **Insert** $([a, b])$: adds the interval $[a, b]$ to your data structure (if it does not exist yet).
3. **Delete** $([a, b])$: removes the interval $[a, b]$ from your data structure (if it exists).
4. **Intersects** $([a, b])$: checks whether your data structure contains at least one interval that has a non-empty intersection with the interval $[a, b]$.

Solution: Use an AVL tree where each interval $[a, b]$ is stored in a node with the key (a, b) . The keys are compared lexicographically to each other, that is, first by the left endpoint of the intervals, and then, if they are the same, by the second endpoint.

This already enables us to perform the first three operations (**Search**, **Insert** and **Delete**) in $O(\log n)$ time.

To be able to do also the **Intersects** operation efficiently, we augment the tree by storing in each node v the right-most endpoint of any interval stored in the subtree rooted at v . We store it in a new attribute called **max_end**. One can easily show that during the other operations this attribute can be updated efficiently as it can change only in nodes along the path from the root to the respective node (that we searched, inserted or deleted). Hence, the update is within the running time of the other operations.

Our algorithm for **Intersects** $([a, b])$ uses

1. the observation: $[a, b]$ intersects an interval $[start, end]$ if and only if $start \leq b$ and $end \geq a$.
2. the invariant: if there is some interval intersecting $[a, b]$, then it is contained in the current subtree.

```

v = root
while still no answer and v ≠ NONE and v.max_end ≥ a do
    if [a, b] intersects the interval of v then
        return True
    else if [a, b] lies completely to the left of the interval of v then
        v = v.left_son
    else // [a, b] lies completely to the right of the interval of v
        if v.left_son.max_end ≥ a then
            // [a, b] intersects an interval that start to the left of v.
            return True
        else
            v = v.right_son
return False

```
